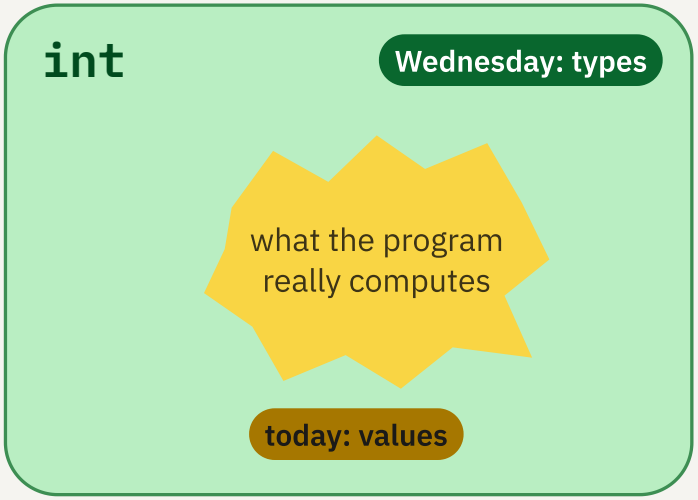
Lecture 10 · Mon 11/2/2026

CS160 Compilers — Fall 2026

Types and Values: What a Program Does.

Yu Feng

Last time, and today.



Wednesday: we cannot prove programs correct, so we check an abstraction of them instead — the green region. Types are that abstraction; the rules `Γ ⊢ e : T` compute them; `run.py` came out well-typed.

Today, in two parts. First, finish the checker: its code, and the one thing it cannot see. Then the yellow side: rules of the same shape that say what a program *does* — what `run.py` actually prints, and why.

At the end the two are put together: the value tree sits inside the type tree, node for node — Wednesday's *sound*, on a real program.

The checker, as code.

typecheck.py

```
def check_expr(e, env):
    """Γ ⊢ e : T.  Returns T and records it on the node."""
    match e:
        case ast.Constant(value=bool()):                # [bool-true] / [bool-false]   (before int: a Python bool is an int)
            t = BOOL
        case ast.BinOp(left=l, op=op, right=r) if isinstance(op, ARITH):   # [arith]
            expect(l, INT, env); expect(r, INT, env)
            t = INT
        case ast.Call(func=ast.Name(id=f), args=args):                     # [invoke]
            _, params, ret = env.lookup(f, e)
            if len(args) != len(params):
                err(e, f"'{f}' expects {len(params)} argument(s), got {len(args)}")
            for a, p in zip(args, params): expect(a, p, env)
            t = ret
        ...
    e.ty = t                                            # annotate: codegen reads this later
    return t

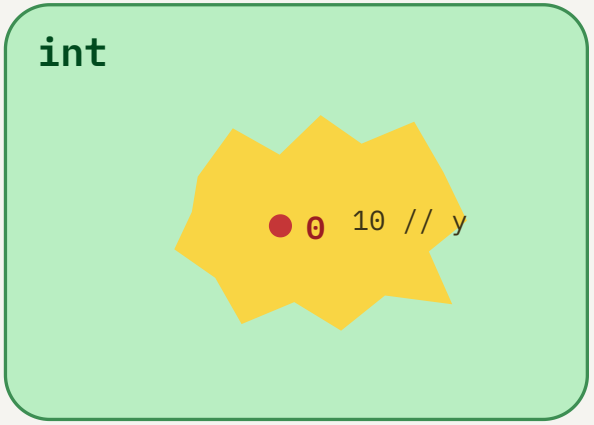
def err(node, msg):
    raise ChocoPyTypeError(f"line {node.lineno}: {msg}")
```

annotate

write the derived type onto the AST node (`e.ty = t`). The tree that leaves the checker is the tree that came in, plus these labels; `print(e)` picks `print_bool` or `print_int` by reading `e.ty`.

One `case` per rule of Wednesday, in slide order; `expect(e, T, env)` is one premise. The recursion builds Wednesday's slide-17 tree top-down, and the first node where a premise fails raises with its line number. That is all of PA3's checker. On paper Γ and R are written separately so each rule shows which it uses; in the code they travel together inside the one `env`.

What the checker cannot see.



divz.py · verified: the checker accepts it; the program exits with runtime error 1

```
y: int = 0
print(10 // y)        # [arith]: int // int is int.  Accepted.
                      # At run time: division by zero.
```

runtime error

an error the rules of the *language* define as happening when the program runs — division by zero, index out of bounds, `len(None)`. Not a crash, not a bug in the compiler: a defined outcome.

The type `int` contains `0`, and `int // int` is an `int`, so the checker is right to accept this: it works on the green region and `0` is in it. What happens when the yellow point is actually `0` is not a question about types.

To say what *does* happen — here, and for every other program — we need the rules for running a program: the yellow half of the picture.

`interp.py`, written as rules.

Wednesday · types

expression $\Gamma \vdash e : T$

statement $\Gamma, R \vdash s$



today · values

expression $G, E, S \vdash e : v, S'$

statement $G, E, S \vdash s : S', R'$

```
eval_expr(e, G, E, S) -> (v, S')
exec_stmt(s, G, E, S) -> (S', R')
```

G
the **global environment**: where top-level names live. A Latin G, not Wednesday's Γ: Γ held types, G holds locations

E
the **local environment**: the parameters and locals of the current call

S
the **store**: what every memory location holds, plus the output printed so far

v
the value produced

S'
the store *afterwards*

R'
statements only: the value a `return` is carrying out, or `None`

E(x)
the location `x` names — in `E` if local, in `G` otherwise

The next seven slides are the interpreter you used as the oracle in PA1 and PA2 — `interp.py` — written as rules. Each rule is one of its `case`s: the premises are the recursive calls, the conclusion is what the call returns, and the store is the memory it updates. The manual's §6 is these rules; the Python file is their translation.

Two judgments, split exactly as Wednesday's: expressions had $\Gamma \vdash e : T$ and produce a value; statements had $\Gamma, R \vdash s$ and produce only effects — a new store, and possibly a `return` on its way out. Same bars, read the same way; Wednesday's conclusion was a type, today's is a value and a store.

Big-step: one judgment goes from an expression straight to its final value. Rules of this kind are an **operational semantics**: a language defined by how it runs, rather than by a paragraph of English.

Constants and variables.

$$\frac{}{G, E, S \vdash n : n, S}\ \text{E-Const} \qquad \frac{}{G, E, S \vdash x : S(E(x)), S}\ \text{E-Var}$$

Wednesday: `[int]` said `3 : int`, `[var-read]` said `x : Γ(x)`. Today: a constant means itself; a variable means what its location holds — `E(x)` is the location, `S(E(x))` what is in it. The same `S` comes out: neither rule prints, writes or fails.

+, *, <.

rules · E-Mul is the same as E-Add, with ×

$$\frac{G, E, S \vdash e_1 : v_1, S_1 \qquad G, E, S_1 \vdash e_2 : v_2, S_2}{G, E, S \vdash e_1 + e_2 : \mathrm{wrap}(v_1 + v_2), S_2}\ \text{E-Add}$$

$$\frac{G, E, S \vdash e_1 : v_1, S_1 \qquad G, E, S_1 \vdash e_2 : v_2, S_2}{G, E, S \vdash e_1 < e_2 : (v_1 < v_2), S_2}\ \text{E-Lt}$$

Subscripts are names: e_1 the first operand, v_1 its value, S_1 the store it left behind.

Compare with `[arith]`: same two premises, one per operand; the conclusion computes the value instead of naming a type. And like `[arith]`, written for two *integers*: no rule anywhere says what + or < does with an `int` beside a `bool`. Slide 12 uses that gap.

Follow the stores: the first premise turns S into S_1, the second *starts from* S_1. The rule has said which operand goes first — slide 13. `wrap`: ChocoPy's `int` is 32-bit; the reference interpreter wraps past the range, and so must your compiler.

run.py, line 4: the call.

Wednesday's program

```
x: int = 3
def inc(a: int) -> int:
    return a + 1
print(inc(x) < 5)
```

G = {x ↦ l_x, inc ↦ its body}, S = {l_x ↦ 3}, E = {} at top level

↦

maps to: S = {l_x ↦ 3} says location l_x holds 3

fresh location

a location nobody uses yet, handed to the parameter: that is why each call has its own a — Lecture 8's frame, as a rule

R'

S-Return fills it; E-Call takes it as the call's value. Expressions have no R' slot at all — a return never leaks past the call

rule · for a function of one parameter a

$$\frac{G, E, S \vdash e : v, S_1 \qquad l \text{ fresh}, \quad S_a = S_1[l \mapsto v], \quad E_a = \{a \mapsto l\} \qquad G, E_a, S_a \vdash \text{body of } f : S', r}{G, E, S \vdash f(e) : r, S'} \text{ E-Call}$$

steps · click for the same four steps as a tree

the four steps for inc(x):

1. the argument: x → 3 — E-Var: S(E(x)) = S(l_x) = 3
2. a new location for the parameter: l_a fresh; S_a = S[l_a ↦ 3]; E_a = {a ↦ l_a}
3. the body, in E_a: a + 1 → 3 + 1 → 4; return hands 4 out as R' — E-Add, S-Return
4. the call's value is what return handed out: inc(x) → 4; the store is S again

Wednesday's tree for inc(x) ended in int; this one ends in 4. Same shape, same leaves; the only addition is the store, which comes out unchanged because nothing printed or wrote.

run.py, line 4: the comparison and the print.

$$\frac{G, E, S \vdash \texttt{inc(x)} : 4, S \quad \text{(slide 8)} \qquad G, E, S \vdash 5 : 5, S}{G, E, S \vdash \texttt{inc(x)} < 5 : \texttt{True}, S}\ \text{E-Lt}$$

$$\frac{G, E, S \vdash \texttt{inc(x)} < 5 : \texttt{True}, S}{G, E, S \vdash \texttt{print(inc(x)} < 5\texttt{)} : S[\texttt{output} \leftarrow \texttt{"True"}], \texttt{None}}\ \text{E-Print}$$

E-Print
the value rule for Wednesday's `[print]`: evaluate the argument, add one line to the output. A statement: its outputs are the new store and `R' = None` — no value, nothing returning

S[output ← "True"]
the store with one more line of output — the only change this program ever makes to the store

---

Three statements, three derivations, like Wednesday: line 1 stores 3 at a fresh l_x; the `def` stores the body under `inc`; line 4 is this tree. The program's meaning is the final store: one line of output, `True`. That is what `interp.py` prints and what your compiler must print.

`if` and `while`.

S-IfFalse: c gives False; run t_1 … tm instead

$$\frac{G, E, S \vdash c : \text{True}, S_1 \qquad G, E, S_1 \vdash s_1 \ldots sn : S_2, R'}{G, E, S \vdash \text{if c: } s_1 \ldots sn \text{ else: } t_1 \ldots tm : S_2, R'}\ \text{S-IfTrue}$$

$$\frac{G, E, S \vdash c : \text{False}, S_1}{G, E, S \vdash \text{while c: s} : S_1, \text{None}}\ \text{S-WhileFalse}$$

the loop

$$\frac{G, E, S \vdash c : \text{True}, S_1 \qquad G, E, S_1 \vdash s : S_2, \text{None} \qquad G, E, S_2 \vdash \text{while c: s} : S_3, R'}{G, E, S \vdash \text{while c: s} : S_3, R'}\ \text{S-WhileTrue}$$

`if`: evaluate the condition once, then run *exactly one* branch — the other contributes nothing, not even its side effects. Wednesday's [`if-elif-else`] checked *both*. That asymmetry is Lecture 9's false positive, from the other side: the line in `falsepos.py` the checker rejected is a line these rules never reach.

`while`: two rules, one for each outcome of the test. In `S-WhileTrue` the last premise is the `while` itself, on the store the body left behind: that recursion *is* the loop, one round per use of the rule.

Even `S-WhileFalse` returns S_1, not S: testing the condition may itself print or write.

Runtime errors are rules; undefined behavior is the absence of one.

$$\frac{G, E, S \vdash e_1 : v_1, S_1 \quad G, E, S_1 \vdash e_2 : v_2, S_2 \quad v_2 \neq 0}{G, E, S \vdash e_1 \,//\, e_2 : v_1 \div v_2, S_2}\ \text{E-Div}$$

$$\frac{G, E, S \vdash e_1 : v_1, S_1 \quad G, E, S_1 \vdash e_2 : \mathbf{0}, S_2}{G, E, S \vdash e_1 \,//\, e_2 : \mathbf{error}, S_2}\ \text{E-DivZero}$$

error

an outcome the rules *name*: evaluation stops, exit status 1, and S_2 is the store it stopped in — everything printed before is out, nothing after ever is. Slide 4's `divz.py` is this rule firing.

undefined behavior

no rule applies, so the language promises nothing: any output is consistent with the rules, and the compiler may assume it never happens. C's `x / 0` and reading past an array are this; ChocoPy has a rule for both.

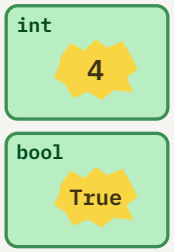
The manual lists exactly the runtime errors an implementation must produce (§6): bad argument to `print`/`len`, division by zero, index out of bounds, operation on None, out of memory. Each has a rule; nothing else may go wrong. Lecture 1's `average.c` printed 25 at `-O0` and 0 at `-O2` (verified) — both correct, because C has no rule to break.

Soundness: a well-typed program never gets stuck.

Wednesday · types · Lecture 9, slide 17

$$\frac{\Gamma \vdash x : \texttt{int}}{\Gamma \vdash \texttt{inc}(x) : \texttt{int}}\ [\text{invoke}] \qquad \Gamma \vdash 5 : \texttt{int}$$

$$\frac{\Gamma \vdash \texttt{inc}(x) : \texttt{int} \qquad \Gamma \vdash 5 : \texttt{int}}{\Gamma \vdash \texttt{inc}(x) < 5 : \texttt{bool}}\ [\text{int-compare}]$$



today · values · slides 8–9

$$\frac{\texttt{G, E, S} \vdash x : \texttt{3}, \texttt{S}}{\texttt{G, E, S} \vdash \texttt{inc}(x) : \texttt{4}, \texttt{S}}\ \text{E-Call} \qquad \texttt{G, E, S} \vdash 5 : \texttt{5}, \texttt{S}$$

$$\frac{\texttt{G, E, S} \vdash \texttt{inc}(x) : \texttt{4}, \texttt{S} \qquad \texttt{G, E, S} \vdash 5 : \texttt{5}, \texttt{S}}{\texttt{G, E, S} \vdash \texttt{inc}(x) < 5 : \texttt{True}, \texttt{S}}\ \text{E-Lt}$$

stuck

a node in a value derivation where *no* rule applies. Run Wednesday's `run_bad.py` without checking it: `inc(x)` gives 4, `True` gives `True`, and now `4 < True` — `E-Lt` compares two integers, and no rule says anything about an `int` beside a `bool`. Not a runtime error, which is a rule — a gap

soundness, restated

a run has three possible endings: a value inside the predicted type, a named `error`, or no ending at all. If a type derivation exists, those three are the only ones — the value derivation never jams at a missing rule.

Left tree against right tree, node for node, on `run.py`: the type is a *prediction*, made without running anything, and at every node the value falls inside it — `x`: 3 in `int`, `inc(x)`: 4 in `int`, the comparison: `True` in `bool`. The prediction is never wrong.

And `run_bad.py`'s value derivation jams at exactly the node where Wednesday's type derivation wrote STUCK (Lecture 9, slide 18). Type-stuck at a node predicts value-stuck at that node — so a *full* type derivation leaves the value side no node to jam at. That is the theorem, in one picture.

What it buys: codegen emits `add i32` and `icmp slt` with *no tags and no runtime checks* — a ChocoPy `int` is four raw bytes — safe only because a checked program never brings a `bool` to `E-Lt`. Python inspects a tag at every `+`, at run time; our checker does it once, before.

Evaluation order is written in the rules.

$$\frac{\textcircled{1}\ G, E, S \vdash e_1 : v_1, S_1 \qquad \textcircled{2}\ G, E, S_1 \vdash e_2 : v_2, S_2}{G, E, S \vdash e_1 + e_2 : \mathrm{wrap}(v_1 + v_2), S_2}\ \text{E-Add}$$

$$S \xrightarrow{①} S_1 \xrightarrow{②} S_2$$

order.py · verified

```
def f() -> int:
    print(1)
    return 10

def g() -> int:
    print(2)
    return 20

print(f() + g())
```

interpreter `1 2 30` compiled `1 2 30` compiled, the two compile_expr lines in BinOp swapped **`2 1 30`**

Evaluation order: when a rule has two premises, which is settled first? The rule itself says: ② starts from S_1, the store ① produced, so ① must be settled first. Swap the two and you have written a *different rule*. ChocoPy fixes it — left to right — in the subscripts on the stores; C leaves it unspecified.

`2 1 30` is an output no derivation produces. Lecture 4's oracle catches the swapped compiler in one run; the rule is what says it was wrong. This is the one thing in today's rules that the type rules know nothing about and your codegen must get right.

What a compiler may change.

times0.py · verified · prints 7 then 0

```
def tick() -> int:
    print(7)
    return 3

print(tick() * 0)
```

side effect

anything code does besides producing a value: printing, stopping with an error, changing something the rest of the program can see — in the rules, $S_1 \neq S$

observationally equivalent

two programs nobody can tell apart from outside: same output, same errors, same order.

May a compiler rewrite `tick() * 0` to `0`? `E-Mul` settles `tick()` first; its print is the change $S \to S_1$; rewriting drops the 7. Invalid. `x * 0 → 0` for a variable `x` *is* valid: `E-Var` returns its store unchanged, so nothing is lost. This is the test every optimization in Lectures 12–14 must pass.

Exercise: evaluate `inc(x) * x`.

deriv.py · verified · prints 6

```
def inc(a: int) -> int:
    return a + 1

x: int = 2
print(inc(x) * x)
```

$G = \{x \mapsto l_x\}, S = \{l_x \mapsto 2\}$. Derive $G, E, S \vdash \texttt{inc(x) * x} : 6, S$, labelling every rule. Then: what does Wednesday's tree say at each node?

Wednesday; PA4.

Wednesday: where a list lives, how `xs[i]` is compiled with its bounds check, the two literals with no type of their own (`[]` and `None` take the type of the place they are stored into — and the manual's $\leq_a$ encoding of that, shown once), and the rules for index-out-of-bounds and `None` — slide 11's list, two more entries.

PA4 (out today, due Fri 11/13): lists and strings. The handout's literal rule and the return-annotation decree are the checker's half — Wednesday's lecture shows both; the interpreter you are given already has your handout's rules (page 1), and the hidden tests were generated from it.

